

# Phatcharadanai Tangoan

Full Stack Developer — Next.js · React · TypeScript

## ABOUT

Computer Engineering graduate (2026) building full-stack web applications with a frontend focus. Four-month co-operative internship at Trinity Securities, a Thai securities firm, owning frontend and backend of internal tools. Since graduating I have shipped two deployed apps end to end — a containerized Kanban board, and a real-time donation platform with its own WebSocket service and payment webhooks.

## SKILLS

### Languages

TypeScript, JavaScript, Python, SQL, HTML/CSS

### Frontend

React, Next.js (App Router), Tailwind CSS, dnd-kit, Framer Motion, Recharts

### Backend

Node.js, Next.js API Routes, Prisma ORM, NextAuth, REST APIs, JWT, WebSocket (ws)

### Databases

PostgreSQL (Neon), MySQL, Redis

### Testing

Vitest, GitHub Actions CI

### DevOps & Tools

Docker (multi-stage), Git, GitHub Actions, Railway, Vercel, Cloudflare R2

## EDUCATION

### Mae Fah Luang University

Bachelor of Engineering (Computer Engineering)  
Graduated 2026 · GPA 3.45

## LANGUAGES & OTHER

**Thai** — Native

**English** — Read technical documentation and English codebases daily.

Military service obligation completed (Ror Dor).

## EXPERIENCE

### Web Developer — Co-operative Internship

Jan – Apr 2026

Trinity Securities Co., Ltd. · Silom, Bangkok

- Built **Tickmatch** end to end — an internal tool tracking how many times a stock ticks upward during a trading day. Python backend ingests market data files and detects price changes; frontend serves a live table with Daily and 1-minute charts. **Redis** for the data pipeline and caching, **Docker** for a reproducible environment.
- Led the frontend of a stock **News Platform** (Next.js + TypeScript) — watchlists, Premium membership, real-time Telegram alerts — and built the initial backend API structure; the AI sentiment-analysis service was owned by a senior engineer.
- Delivered **Reweb Trinity**, a Next.js + TypeScript prototype redesigning the company website for a younger investor audience.

## PROJECTS

### DONATR — Streamer Donation Platform

Jul – Aug 2026 · [Live](#)

Next.js 16 · TypeScript · Node.js (ws) · Prisma · PostgreSQL · Omise · Railway · Vercel

- I wrote the **WebSocket service** behind the alert overlay myself (Node + **ws**) instead of using a hosted one, so a donation reaches the streamer's **OBS** within about a second. The overlay opens with a long-lived token, trades it for a ticket that lasts 60 seconds and can only be used once, and reconnects on its own when the connection drops.
- Donations are paid through **Omise** in test mode. When the payment webhook arrives its signature is verified and the donation is settled in a single **guarded update** — which is what makes a duplicate webhook, or a retry, unable to fire the same alert twice.
- Azure Speech** reads the donor's message aloud over the alert. It is synthesised once and cached, so replaying an alert costs nothing. The site is a demo and never takes real money; **319 tests** run in GitHub Actions.

### Taskboard — Kanban Collaboration App

May – Jun 2026 · [Live](#) · [GitHub](#)

Next.js 16 · React · Prisma · PostgreSQL · Tailwind v4 · dnd-kit · Docker

- A board used to load in several round-trips, so I folded a redundant per-request access check into the main query and it now loads in a **single JOIN**.
- File uploads go straight to **Cloudflare R2** through presigned URLs, with size limits, filename sanitization and a MIME allow-list enforced on the server, and orphaned objects cleaned up when a card is deleted.
- Sign-up is atomic: a Prisma **\$transaction** commits the user and its one-time code together, or rolls both back. Sign-in supports credentials with OTP email verification and Google OAuth through **NextAuth**, and viewer/editor roles are checked on the server.
- Cards and columns are reordered by drag-and-drop with optimistic UI, alongside checklists, comments, labels and due dates, and an accessibility pass over the dialogs and touch targets.
- It ships as a multi-stage **Docker** image to Railway with Neon PostgreSQL — debugging a Prisma musl/OpenSSL engine failure and a production login redirect loop on the way — and is guarded by 28 **Vitest** tests in **GitHub Actions**.

### Suicide Risk Surveillance System — Senior Project

[GitHub](#)

Next.js · PostgreSQL · Prisma · Python (scikit-learn) · Leaflet · Recharts

- A team project, and a teammate owned the Random Forest model; I built the web application. Staff upload a CSV, which triggers a Python script that engineers the features, runs the prediction and writes the results back to PostgreSQL, and the dashboard shades every district by risk tier on a **Leaflet** map with drill-down and filterable breakdowns.